

Space-Efficient Indexes for Uncertain Strings

Esteban Gabory¹, Chang Liu², Grigorios Loukides³, Solon P. Pissis^{1,4}, and Wiktor Zuba¹

¹CWI, Amsterdam, The Netherlands

²Zhejiang University, Zhejiang, China

³King's College London, London, UK

⁴Vrije Universiteit, Amsterdam, The Netherlands

Abstract—Strings in the real world are often encoded with some level of uncertainty, for example, due to: unreliable data measurements; flexible sequence modeling; or noise introduced for privacy protection. In the *character-level uncertainty model*, an *uncertain string* X of length n on an alphabet Σ is a sequence of n probability distributions over Σ . Given an uncertain string X and a weight threshold $\frac{1}{z} \in (0, 1]$, we say that pattern P occurs in X at position i , if the product of probabilities of the letters of P at positions $i, \dots, i + |P| - 1$ is at least $\frac{1}{z}$. While indexing standard strings for online pattern searches can be performed in linear time and space, indexing uncertain strings is much more challenging. Specifically, the state-of-the-art index for uncertain strings has $\mathcal{O}(nz)$ size, requires $\mathcal{O}(nz)$ time and $\mathcal{O}(nz)$ space to be constructed, and answers pattern matching queries in the optimal $\mathcal{O}(m + |\text{Occ}|)$ time, where m is the length of P and $|\text{Occ}|$ is the total number of occurrences of P in X . For large n and (moderate) z values, this index is completely impractical to construct, which outweighs the benefit of the supported optimal pattern matching queries. We were thus motivated to design a space-efficient index at the expense of slower yet competitive pattern matching queries. We show that when we have at hand a lower bound ℓ on the length of the supported pattern queries, as is often the case in real-world applications, we can slash the index size and the construction space roughly by ℓ . In particular, we propose an index of $\mathcal{O}(\frac{nz}{\ell} \log z)$ expected size, which can be constructed using $\mathcal{O}(\frac{nz}{\ell} \log z)$ expected space, and supports very fast pattern matching queries in expectation, for patterns of length $m \geq \ell$. We have implemented and evaluated several versions of our index. The best-performing version of our index is up to two orders of magnitude smaller than the state of the art in terms of both index size and construction space, while offering faster or very competitive query and construction times.

I. INTRODUCTION

A large portion of the data feeding real-world database systems, including bioinformatics systems [73], Enterprise Resource Planning (ERP) systems [67], or Business Intelligence (BI) systems [84], is textual; namely, *sequences* of letters over some alphabet (also known as *strings*). This happens because strings can easily encode data arising from different sources such as: sequences of nucleotides read by DNA sequencers (e.g., short or long DNA reads) [41]; natural language text generated by humans (e.g., description or comment fields) [50]; id's generated by software (e.g., URLs, email addresses, or IP addresses) [21]; or discretized measurements generated by sensors (e.g., RSSI, EEG or EMG data) [11], [83].

Given the ever increasing size of string data, it is crucial to represent them in a concise form but also to *simultaneously* allow efficient pattern searches. This goal is formalized by the

classic *text indexing* problem [31]: preprocess a string T of length n over an alphabet Σ of size σ , known as the *text*, into a data structure that supports pattern matching queries; i.e., report the set Occ of all $|\text{Occ}|$ positions in T where an occurrence of a string P , known as the *pattern*, begins.

In text indexing we are interested in four efficiency measures [16], [56]: (i) How much space does the final index occupy for a string T of length n (the *index size*)? (ii) How fast can we answer a query P of length m (the *query time*)? (iii) How much working space do we need to construct the index (the *construction space*)? (iv) How fast can we construct the index (the *construction time*)? The classic *suffix tree* index [85] has size linear in n and optimal query time $\mathcal{O}(m + |\text{Occ}|)$; the construction time and space are also linear in n [35].

In the real world, strings are often encoded with some level of uncertainty; for example, due to: (i) imprecise, incomplete or unreliable data measurements, such as sensor measurements, RFID measurements or trajectory measurements [10]; (ii) deliberate flexible sequence modeling, such as the representation of a *pangenome*, that is, a collection of closely-related genomes to be analyzed together [80]; or (iii) the existence of confidential information in a dataset which has been distorted deliberately for privacy protection [9].

While there are many practical solutions for text indexing [64], [36], [42], [38], [16] and also for answering different types of queries on various types of uncertain data (see Section VI), *practical indexing schemes* for uncertain strings are rather undeveloped. In response, our work makes an important step towards developing such practical space-efficient indexes.

A. Our Data Model and Motivation

We use the standard *character-level uncertainty model* [48]. An *uncertain string* (or *weighted string*) X of length n on an alphabet Σ is a sequence of n sets. Every $X[i]$, for all $i \in [1, n]$, is a set of $|\Sigma|$ ordered pairs $(\alpha, p_i(\alpha))$, where $\alpha \in \Sigma$ and $p_i(\alpha)$ is the probability that letter α occurs at position i .

Example 1. The table below shows a weighted string X , with $n = 6$ and $\Sigma = \{A, B\}$, represented as a $|\Sigma| \times n$ matrix. ■

	1	2	3	4	5	6
A	1	1/2	3/4	4/5	1/2	1/4
B	0	1/2	1/4	1/5	1/2	3/4

The data model of [48] has been employed by many works [13], [59], [69], [39], [60], [57]. In bioinformatics,

for example, weighted strings are known as *position weight matrices* [54]. As in these works, we define the *occurrence probability* of pattern $P = \text{ABA}$ at position 3 in X of Example 1 as $3/4 \times 1/5 \times 1/2 = 3/40$ (shown in bold).

Weighted Indexing. The *Weighted Indexing* problem is defined as follows [14]: Given a weighted string X of length n on an alphabet Σ of size σ and a weight threshold $\frac{1}{z} \in (0, 1]$, preprocess X into a *compact* data structure (*the index*) that supports *efficient* pattern matching queries; i.e., report all positions in X where P occurs with probability at least $\frac{1}{z}$.

State of the Art. The indexing problem on weighted strings has attracted a lot of attention by the theory community [46], [14], [20], [18], [20], [17], [23], culminating in the following bounds [18], [17]: there exists an index of $\mathcal{O}(nz)$ size supporting optimal $\mathcal{O}(m + |\text{Occ}|)$ -time queries for any pattern of length m ; it can be constructed in $\mathcal{O}(nz)$ time using $\mathcal{O}(nz)$ space. We call this index the *weighted suffix tree* (WST).

Our Motivation. Although these bounds are very appealing from a theoretical perspective, from a practical perspective, $\mathcal{O}(nz)$ size and construction space are *prohibitive* for large-scale datasets. Say we have an input weighted string of length $n = 10^9$ bytes, that $z = 100$, and that the constant in $\mathcal{O}(nz)$ is something small, like 20 bytes. Then we need around 2TBs of RAM to store the index for an input of 1GB! *We were thus motivated to seek space-query time trade-offs for indexing weighted strings.* In particular, we seek a parameterized version of *Weighted Indexing* for which we can have indexes of size smaller than $\mathcal{O}(nz)$. Ideally, we would also like to construct these smaller indexes using less than $\mathcal{O}(nz)$ space. A good such input parameter is a lower bound ℓ on the length m of any queried pattern. It is arguably a reasonable assumption to know ℓ in advance. For instance, in bioinformatics [47], [86], [61], the length of sequencing reads (patterns) ranges from a few hundreds to 30,000 [61]. Even when at most k errors must be accommodated for matching, at least one out of $k + 1$ fragments must be matched exactly. In natural language processing, the queried patterns can also be long [82]. Examples of such patterns are queries in question answering systems [43], *description queries* in TREC datasets [19], [15], and representative phrases in documents [65]. Similarly, a query pattern can be long when it encodes an entire document (e.g., a webpage in the context of deduplication [44]), or machine-generated messages [49].

B. Our Techniques and Results

Let us first recall WST, the state-of-the-art index. In [18], Barton et al. showed that, for any weight threshold $\frac{1}{z}$, a weighted string X of length n can be transformed into a family $\mathcal{S}$ of z standard strings (each of length n), so that a pattern P occurs in X at position i with probability p only if P occurs at position i in $\lfloor p \cdot z \rfloor$ strings from $\mathcal{S}$. The authors have also shown how to index $\mathcal{S}$ using (a modified version of) suffix trees [85] resulting in WST: an index of total size $\mathcal{O}(nz)$ supporting pattern matching queries in the optimal $\mathcal{O}(m + |\text{Occ}|)$ time. A more space-efficient, array-based version was also presented in [23]; it is known as the *weighted suffix array* (WSA).

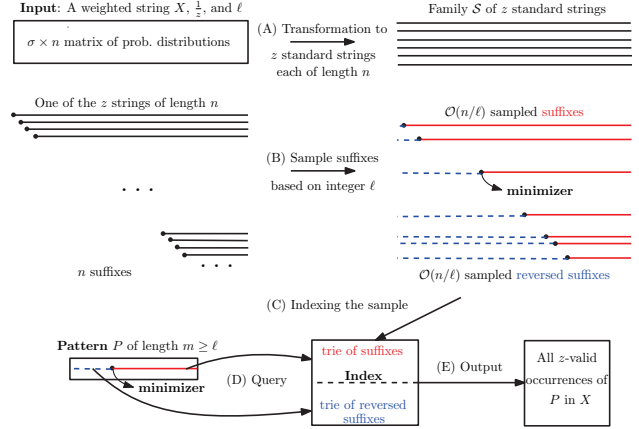


Fig. 1: An informal overview of our techniques: Given a weighted string X of length n over an alphabet of size σ , a weight threshold $\frac{1}{z}$, and an integer ℓ , we (A) use the algorithm from [18] to construct a family $\mathcal{S}$ of z standard strings, each of length n . (B) For each such string, we consider all of its n suffixes and sample them for the given integer ℓ using the minimizers mechanism [72], [74]. These suffixes imply a set of $\mathcal{O}(n/\ell)$ suffixes and a set of $\mathcal{O}(n/\ell)$ reversed suffixes in expectation. (C) We then index these suffixes in two suffix trees [85], which we link using a 2D grid, so as to answer pattern matching queries for patterns of length at least ℓ . (D) When such a pattern P of length m is given, we find its leftmost minimizer, which implies a suffix and a reversed suffix of P , and query those using our index. Our index efficiently merges the partial results (i.e., occurrences of suffixes and reversed suffixes); and (E) outputs all z -valid occurrences of P in X . The size of the resulting index is $\mathcal{O}(\frac{nz}{\ell} \log z)$. The extra multiplicative $\log z$ factor comes from our representation of the edge labels in the suffix trees.

Here we make the following three main contributions (see Fig. 1 for an informal overview of our techniques):

- 1) We show how to slash the size of both WST and WSA roughly by ℓ , while still supporting very fast queries in expectation for any pattern P of length $m \geq \ell$, by combining the *minimizers* sampling mechanism [72], [74], *suffix trees* [85], several combinatorial and probabilistic arguments, and a *geometric data structure* (2D grid) [63]. The size of the resulting index is $\mathcal{O}(\frac{nz}{\ell} \log z)$. The extra multiplicative $\log z$ factor comes from a new way to encode the edge labels in the suffix trees. The main novelty of our approach is the combination of minimizers and the new edge encoding allowing us to delete the z strings after the construction, thus resulting in a significantly smaller index size for large ℓ values.
- 2) Our technique still requires us to first construct the family $\mathcal{S}$ of the z strings, which in any case gives an index with $\mathcal{O}(nz)$ construction space. The challenge is how to construct the index *without* constructing the z strings. To achieve this, we develop a fast, highly non-

trivial, algorithm for constructing the *same* index *without* generating $\mathcal{S}$ explicitly. Our new algorithm samples an implicit representation of $\mathcal{S}$ using minimizers outputting the final index directly. The construction space of the resulting index matches its size: $\mathcal{O}(\frac{nz}{\ell} \log z)$. This is the most technically involved result of our paper.

- 3) Following the different paradigms of suffix trees [85] and suffix arrays [52] in the classic setting of standard strings, we have implemented tree and array-based versions of our index underlying Contributions 1 and 2. The results show that our indexes are up to *two orders of magnitude smaller* than the state of the art in terms of *both* index size and construction space. For instance, for indexing 1,432 bacterial samples, with $\ell = 1024$ and $z = 128$, which are reasonable in applications, our space-efficient index has size 640MBs and needs only 772MBs of memory to be constructed, while WSA has size 7GBs and needs 32GBs of memory to be constructed and WST has size 126GBs and needs 241GBs of memory to be constructed! Our results also show that our array-based indexes outperform the tree-based ones, offering *very competitive query times and construction times* to those of the state of the art.

C. Paper Organization

In Section II, we provide the necessary background. In Section III, we present our index. In Section IV, we present the space-efficient algorithm for constructing our index. In Section V, we present a more practical algorithm for querying our index. In Section VI, we discuss related work. In Section VII, we provide an extensive experimental evaluation of our algorithms. We conclude in Section VIII with a discussion on limitations and future work.

II. PRELIMINARIES AND PROBLEM DEFINITION

Strings. An *alphabet* Σ of size $\sigma = |\Sigma|$ is a nonempty set of elements called *letters*. By Σ^k we denote the set of all length- k strings over Σ . By ε we denote the *empty string* of length 0. For a *string* $S = S[1] \dots S[n]$ over Σ , by $n = |S|$ we denote its *length*. The fragment $S[i..j]$ of S is an *occurrence* of the underlying *substring* $P = S[i] \dots S[j]$. We also say that P occurs at *position* i in S . A *prefix* of S is a substring of S of the form $S[1..j]$ and a *suffix* of S is a substring of S of the form $S[i..n]$. Given a string $S = S[1] \dots S[n]$, its *reverse* is the string $S^r = S[n] \dots S[1]$. For any two strings S_1 and S_2 of the same length, we define their *Hamming distance* $d_H(S_1, S_2)$ as their total number of mismatching positions.

Sampling. Given a fixed pair of positive integers (ℓ, k) s.t. $\ell \geq k$, we call a function $f: \Sigma^\ell \rightarrow [1, \ell - k + 1]$ that selects the starting position of a length- k fragment, for any string of length ℓ , an (ℓ, k) -*local scheme*. We call the set $\mathcal{M}_f(S) = \{i + f(S[i..i + \ell - 1]) - 1 \mid 1 \leq i \leq |S| - \ell + 1\}$, for an (ℓ, k) -*local scheme* f on a string S , the *set of selected indices*. An (ℓ, k) -*minimizer scheme* is an (ℓ, k) -local scheme that selects the position of the leftmost occurrence of the smallest length- k substring, for a fixed k and a fixed order on Σ^k . In that

case, we call *minimizers* the selected indices [72], [74]. The minimizer scheme can be based on a lexicographic order.

Example 2. Let $S = \text{ABAABB}$, $\ell = 4$, and $k = 2$. We obtain $\mathcal{M}_f(S) = \{3\}$ because $S[3..4] = \text{AA}$ is the lexicographically smallest length-2 substring in all length-4 fragments of S . ■

The minimizer scheme can also be specified by a hash function, e.g., Karp-Rabin fingerprints [53].

Definition 1. Let f be an (ℓ, k) -minimizer scheme. The *specific density* of f on S is the value $|\mathcal{M}_f(S)|/|S|$. The *density* of f is the expected specific density on a sufficiently long random string (with letters chosen independently at random).

Lemma 1 ([90]). *The density of an (ℓ, k) -minimizer scheme on alphabet Σ with $k \geq \log_{|\Sigma|} \ell + c$ is $\mathcal{O}(\frac{1}{\ell})$, for some $c = \mathcal{O}(1)$.*

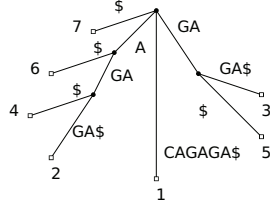
Weighted Strings. A *weighted string* X of length n on an alphabet Σ is a sequence of n sets $X[1], \dots, X[n]$. Every $X[i]$, for all $1 \leq i \leq n$, is a set of $|\Sigma|$ ordered pairs $(\alpha, p_i(\alpha))$, where $\alpha \in \Sigma$ is a letter and $p_i(\alpha)$ is the probability of having α at position i of X . Formally, $X[i] = \{(\alpha, p_i(\alpha)) \mid \alpha \in \Sigma\}$, where for every $\alpha \in \Sigma$ we have $p_i(\alpha) \in [0, 1]$, and $\sum_{\alpha \in \Sigma} p_i(\alpha) = 1$. A letter α *occurs* at position i of a weighted string X if and only if $p_i(\alpha) > 0$. A string U of length m is a *factor* of a weighted string X if and only if it occurs at some starting position i with *occurrence probability* $\mathbb{P}(X[i..i + m - 1] = U) = \prod_{j=1}^m p_{j+i-1}(U[j]) > 0$. Given a *weight threshold* $1/z \in (0, 1]$, we say that factor U is *z-solid* (or *z-valid*) or equivalently that factor U has a *z-solid occurrence* in X at some position i , if $\mathbb{P}(X[i..i + m - 1] = U) \geq 1/z$. When the context is clear, we may simply say *solid* (or *valid*). We say that factor U is *(right-)maximal* at position i of X if U has a solid occurrence at position i of X and no string $U' = U\alpha$, for any $\alpha \in \Sigma$, has a solid occurrence at position i of X . For a weighted string X , a pattern P , and a weight threshold $1/z \in (0, 1]$, $\text{Occ}_{1/z}(P, X)$ is the set of starting positions of valid occurrences of P in X . A *property* Π of a string S is a hereditary collection of integer intervals contained in $[1, n]$.¹ For simplicity, we represent every property Π with an array $\pi[1..|S|]$ such that the longest interval $I \in \Pi$ starting at position i is $[i, \pi[i]]$. Observe that π can be an arbitrary array satisfying $\pi[i] \in [i - 1, n]$, and $\pi[1] \leq \pi[2] \leq \dots \leq \pi[n]$ (where $\pi[i] = i - 1$ means that i is not contained in any interval $I \in \Pi$). For a string P , by $\text{Occ}_\pi(P, S)$ we denote the set of occurrences of P in S such that $i + |P| - 1 \leq \pi[i]$.

Example 3. Consider the string-property pair (S_2, π_2) in Table I. The pattern $P = \text{AAA}$ occurs at position $i = 3$ because $i + |P| - 1 = 3 + 3 - 1 \leq \pi_2[3] = 5$. ■

Let us consider an indexed family $\mathcal{S} = (S_j, \pi_j)_{j=1}^k$ of strings S_j with properties π_j . For a string P and an index i , by $\text{Count}_{\mathcal{S}}(P, i) = |\{j \mid i \in \text{Occ}_{\pi_j}(P, S_j)\}|$ we denote the total number of occurrences of P at position i in the

¹A collection that contains all the subintervals of its elements.

i	1	2	3	4	5	6
S_1	A	A	A	A	A	A
π_1	2	2	3	4	5	6
S_2	<u>A</u>	<u>A</u>	<u>A</u>	<u>A</u>	A	B
π_2	4	4	5	6	6	6
S_3	<u>A</u>	B	<u>A</u>	<u>A</u>	B	B
π_3	4	4	5	6	6	6
S_4	A	B	B	B	B	B
π_4	2	2	3	3	5	6



strings $S_1, \dots, S_{s_k}$ of $\mathcal{S}$ that respect the properties. We say that an indexed family $\mathcal{S} = (S_j, \pi_j)_{j=1}^{\lfloor z \rfloor}$ is a *z-estimation* of a weighted string X of length n if and only if, for every string P and position $i \in [1, n]$, $\text{Count}_{\mathcal{S}}(P, i) = \lfloor \mathbb{P}(X[i..i+|P|-1] = P) \cdot z \rfloor$. The following result has been shown by Barton et al.:

Example 4. For $\frac{1}{z} = \frac{1}{4}$, the weighted string X in Example 1 admits the 4-estimation $\mathcal{S}$ in Table I, given by Theorem 2.

For pattern $P = \text{AB}$ and $i = 1$, we have that $\mathbb{P}(X[i \dots i + |P| - 1] = P) = 1 \cdot 1/2 = 1/2$ and so P occurs in $\text{Count}_S(P, i) = [\mathbb{P}(X[i \dots i + |P| - 1] = P) \cdot z] = [(1/2) \cdot 4] = 2$ strings of the 4-estimation at position 1.

Problem Definition. In our ℓ -WEIGHTED INDEXING problem, we are given a weighted string X of length n over an alphabet Σ , a weight threshold $\frac{1}{z} \in (0, 1]$, and an integer $\ell > 0$, and we are asked to preprocess them in a *compact* data structure (the index) to support the following queries *efficiently*: For any string P of length $m \geq \ell$, report all elements of $\text{Occ}_{\frac{1}{z}}(P, X)$. Other than the *index size* and the *query time*, we seek to minimize the *construction time* and the *construction space*.

We next summarize the crucial notation of our paper.

Σ	Alphabet
σ	Size of the alphabet
X	Weighted string
n	Length of X
$1/z$	Weight threshold
P	Pattern
m	Length of P
ℓ	Lower bound on the length m of P
p_i	Probability distribution at $X[i]$
$X[i]$	i th set $\{(\alpha, p_i(\alpha)) \mid \alpha \in \Sigma\}$ of X
$\mathbb{P}(X[i \dots j] = P)$	Probability $\Pi_{h=0}^{j-i} P[h+1]$
$\mathcal{M}_X$	Minimizers for solid factors of X
k	Length of minimizer substrings
H_X	Heavy string of X
$\mathcal{S}$	Property array $(S_j, \pi_j)_{j=1}^n$
$\pi_i[j] = h$	$S_i[j \dots h]$ is a solid factor
$ \text{Occ} $	Number of occurrences in the output
$\text{Occ}_{1/z}(P, X)$	The z -solid occurrences of P in X
$\mathcal{T}_{\text{succ}}$	Forward minimizer solid factor tree
$\mathcal{T}_{\text{pref}}$	Backward minimizer solid factor tree
$\mathcal{L}_{\text{succ}}(P)$	Leaf nodes in $\mathcal{T}_{\text{succ}}$ after reading P
$\mathcal{L}_{\text{pref}}(P)$	Leaf nodes in $\mathcal{T}_{\text{pref}}$ after reading P
$\mathcal{N}(P)$	Query rectangle for P in the 2D grid

In this section, we describe our index for solving ℓ -WEIGHTED INDEXING. We assume random access to X (we can discard X at the end of the construction). To simplify the analysis we assume X is over an alphabet of size $\sigma = \mathcal{O}(1)$.

Minimizer Solid Factor Trees. Let us fix a weighted string X of length n over an alphabet Σ and a weight threshold $\frac{1}{z}$. We first define a *forward solid factor tree* (resp. *backward solid factor tree*) for X as the suffix tree for the set of maximal solid factors (resp. the set of reversed solid factors) in X . By Theorem 2, we know that each such solid factor appears in a z -estimation of size $\mathcal{O}(nz)$, and therefore both solid factor trees have size $\mathcal{O}(nz)$ as well. This argument also gives a method to construct the solid factor trees [17].

We adapt the solid factor trees to make them more space-efficient for ℓ -WEIGHTED INDEXING by employing minimizer schemes. Let us fix ℓ, k and an (ℓ, k) -minimizer scheme f . We can assume throughout, from Lemma 1, that ℓ and k are chosen so that the density of f (see Def. 1) is $\mathcal{O}(\frac{1}{\ell})$. We then construct a z -estimation $\mathcal{S} = (S_j, \pi_j)_{j=1}^{\lfloor \frac{nz}{\ell} \rfloor}$ of X using Theorem 2 and compute the set $\mathcal{M}_X$ of minimizers from $\mathcal{S}$ respecting the property; namely for $S_j \in \mathcal{S}$ we compute $f(S_j[i..i+\ell-1])$ if and only if $i+\ell-1 \leq \pi_j[i]$.

We represent each minimizer in $\mathcal{M}_X$ by a pair (i, j) , where i is the minimizer position in the string $S_j \in \mathcal{S}$. In the following, we consider $\mathcal{M}_X$ fixed with $|\mathcal{M}_X| = \mathcal{O}(\frac{nz}{\ell})$, as by Lemma 1 there are in expectation $\mathcal{O}(\frac{nz}{\ell})$ minimizers in $\mathcal{S}$.

Based on $\mathcal{M}_X$, we define a *minimizer forward* (resp. *backward*) solid factor tree as a compacted trie containing suffixes of solid factors (resp. of reversed solid factors) *starting* at position i from a string $S_j \in \mathcal{S}$ with $(i, j) \in \mathcal{M}_X$. Each leaf has a minimizer label $(i, j) \in \mathcal{M}_X$ associated to the corresponding suffix. If one same suffix corresponds to several such labels (it occurs at several minimizers from $\mathcal{S}$), we add one copy of the leaf for each such label. Since $|\mathcal{M}_X| = \mathcal{O}(\frac{nz}{\ell})$, the minimizer solid factor trees contain $\mathcal{O}(\frac{nz}{\ell})$ leaves, and therefore nodes.

Still the size of the z -estimation $\mathcal{S}$ is, by definition, always $\Theta(nz)$, which makes the total size of the index $\mathcal{O}(\frac{nz}{\ell}) + \Theta(nz) = \Theta(nz)$. We avoid this by employing the following crucial combinatorial observation on *heavy strings* [58]:

Definition 2. For any weighted string X , we call a *heavy string* H_X of X a string such that $H_X[i]$ is the letter having a largest probability in $X[i]$ (ties are broken arbitrarily).

Example 5. Let X be the weighted string of Example 1; a heavy string of X is $H_X = \text{ABAAAB}$ (the tie at position 2 is broken for B and the tie at position 5 is broken for A). ■

Lemma 3 ([58]). Let H_X be a heavy string of X . For a weight threshold $\frac{1}{z}$ and any z -solid factor U starting at position i and ending at position j of X , $d_H(U, H_X[i..j]) \leq \log_2 z$ holds.

Example 6. Let X be the weighted string of Example 1; with $H_X = \text{ABAAAB}$. If $z = 4$, $\log_2 z = 2$, so no solid factor has more than 2 mismatches with H_X at its occurrence position. The string AABB is not valid at position 1 as it has 3 mismatches with H_X ; indeed, its probability is $\frac{1}{40} < \frac{1}{4}$. The string AAAA is valid at position 1 with probability $\frac{3}{10} > \frac{1}{4}$, and has only one mismatch with H_X . The condition is however not sufficient: ABAB is not valid at position 1 (its probability is $\frac{3}{40} < \frac{1}{4}$), even if it has only one mismatch with H_X . ■

We directly get the following result, which allows us to avoid storing the z -estimation $\mathcal{S}$ explicitly.

Corollary 4. Every solid factor of a weighted string X for a weight threshold $\frac{1}{z}$ can be characterized by an interval of the heavy string H_X plus the information of at most $\log_2 z$ single mismatches. The minimizer solid factor tree can be implemented as a compacted trie whose edges store only that information, which takes $\mathcal{O}(\log z)$ extra space per edge.

We apply Corollary 4 to obtain Lemma 5. Based on this lemma, we construct the minimizer solid factor trees for X .

Lemma 5. The minimizer solid factor trees can be constructed in $\mathcal{O}(nz)$ time using $\mathcal{O}(nz)$ space. Each tree has $\mathcal{O}(\frac{nz}{\ell})$ expected nodes and its expected total size is $\mathcal{O}(\frac{nz}{\ell} \log z)$.²

All omitted proofs can be found in [37].

Exploiting 2D Range Reporting. We explain how to employ a geometric data structure to pair up the leaf nodes corresponding to the same minimizer position from one of the minimizer solid factor tree that we have constructed above to the other.

Let us write $\mathcal{T}_{\text{suff}}$ (resp. $\mathcal{T}_{\text{pref}}$) for the forward (resp. backward) minimizer solid factor tree. We fix an order on the leaves of both $\mathcal{T}_{\text{pref}}$ and $\mathcal{T}_{\text{suff}}$, such that for any node in one of the trees, the set of its descendant leaves forms an interval. This is possible, for example, by sorting the strings corresponding to the leaves in lexicographical order. Via this ordering, we can consider a pair of leaves from $\mathcal{T}_{\text{suff}}$ and $\mathcal{T}_{\text{pref}}$ as a point of a 2D data structure, which we call the *grid*.

We start by some definitions: (1) Given a string P , we denote by $\mathcal{I}_{\text{suff}}(P)$ (resp. $\mathcal{I}_{\text{pref}}(P)$) the (possibly empty) interval of leaves in the subtree obtained by reading P in $\mathcal{T}_{\text{suff}}$ (resp. $\mathcal{T}_{\text{pref}}$). (2) We denote by $\mathcal{N}$ the set of all those points corresponding to pairs of leaves from $\mathcal{T}_{\text{suff}}$ and $\mathcal{T}_{\text{pref}}$ with identical minimizer labels. Each point in $\mathcal{N}$ corresponds to a given minimizer $(i, j) \in \mathcal{M}_X$, and a pair of maximal solid factors in X that can be read from i , both right-wise and left-wise. (3) Given a pair P_1, P_2 of strings, we denote by $\mathcal{N}(P_1, P_2)$ the intersection of the set $\mathcal{N}$ with the rectangle $\mathcal{I}_{\text{suff}}(P_1) \times \mathcal{I}_{\text{pref}}(P_2)$. (4) Given a string P of length $m \geq \ell$, such that $f(P[1.. \ell]) = \mu$, we denote $\mathcal{N}(P[\mu..m], (P[1.. \mu])^r)$ by $\mathcal{N}(P)$.

Lemma 6. For any pattern P of length m , with $n \geq m \geq \ell$, if P is a solid factor in X , then $\mathcal{N}(P)$ is nonempty. In particular, if P has a valid occurrence in X starting at position k then $\mathcal{N}(P)$ contains at least one point having label $(k-1 + f(P[1.. \ell]), j)$ for some $j \in [1, \lfloor z \rfloor]$.

Our index (i.e., $\mathcal{T}_{\text{suff}}$, $\mathcal{T}_{\text{pref}}$, and the grid) solves ℓ -WEIGHTED INDEXING by answering 2D range reporting queries [22]. In the 2D range reporting problem, we are given a set $\mathcal{N}$ of N points to be preprocessed, so that when one gives an axis-aligned rectangle as a query, we report the subset $\mathcal{K}$ of $\mathcal{N}$ such that point $p \in \mathcal{K}$ if and only if the rectangle encloses p . We consider a special case of this problem which suffices for our purposes. In particular, we use the following result.

Lemma 7 (Section 2 of [63]). Let $\mathcal{N}$ be a set of N points coming from pairing two permutations of $[1, N]$. With $\mathcal{O}(N)$ construction time, we can answer 2D range reporting queries in $\mathcal{O}((1+k) \log N)$ time using $\mathcal{O}(N)$ space, where k is the number of points from $\mathcal{N}$ enclosed by the query rectangle.

Even if each occurrence of a pattern can be detected with 2D range reporting queries, the converse is not true: if a

²We claim $\mathcal{O}(nz)$ time and space during our construction because if $\log z > \ell$, we can abort the construction and resort to $\mathcal{O}(nz)$ size.

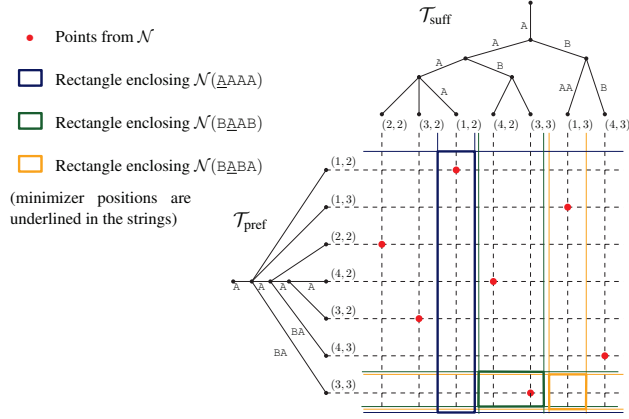


Fig. 3: Our minimizer-based index for the weighted string from Example 1, $\frac{1}{z} = \frac{1}{4}$, and the minimizers from Example 7. $\mathcal{T}_{\text{suff}}$ is the forward minimizer solid factor tree and $\mathcal{T}_{\text{pref}}$ is the backward one. Edges without labels are constructed for readability and mean that the parent and the children nodes correspond to the same string. Each leaf node representing the minimizer position i in string S_j is decorated with (i, j) .

pattern U has a minimizer at position μ and both $U[1.. \mu]$ and $U[\mu.. |U|]$ are solid factors occurring at respective positions k and $k + \mu - 1$, then a corresponding point will be detected with the 2D range reporting queries, *even* if U is not a solid factor itself. In that case, U is by definition a substring of some $S_j \in \mathcal{S}$, but does *not* respect the property. We can simply compute all the points by 2D range reporting, and check naively for false positives by comparing the pattern with X at the positions corresponding to these points. Conversely, one can have several points corresponding to a single occurrence, if the pattern appears in multiple strings in $\mathcal{S}$ at a same position, which could also increase the running time. To control the number of such additional checks (both for false positives and duplicate ones), we give a bound on the expected number of occurrences of a given pattern in the z -estimation $\mathcal{S}$:

Lemma 8. *For any string P chosen uniformly at random from Σ^m , there are $\mathcal{O}(nz/\sigma^m)$ points expected in $\mathcal{N}(P)$.*

Main Result. We arrive at the main result of the section:

Theorem 9. *Let X be a weighted string of length n over an alphabet of size σ , $\frac{1}{z}$ be a weight threshold, and $\ell > 0$ be an integer. After $\mathcal{O}(nz)$ construction time and using $\mathcal{O}(nz)$ construction space, we can construct an index of $\mathcal{O}(n + \frac{nz}{\ell} \log z)$ expected size answering ℓ -WEIGHTED INDEXING queries of length $m \geq \ell$ in $\mathcal{O}(m + (1 + \frac{nz}{\sigma^m}) \log \frac{nz}{\ell})$ expected time.*

Example 7. Let X be the weighted string from Example 1 and $\frac{1}{z} = \frac{1}{4}$. The construction of our index is detailed in Fig. 3, and query rectangles $\mathcal{N}(P)$ (resp. $\mathcal{N}(P')$ and $\mathcal{N}(P'')$) are constructed for patterns $P = \underline{A}AAA$ (resp. $P' = B\underline{A}AB$ and $P'' = B\underline{A}BA$) whose minimizer positions are underlined. The blue rectangle $\mathcal{N}(P)$ contains exactly one point which

corresponds to a substring $AAAA$ in S_2 . This substring corresponds to an occurrence of P at position 1 with probability $1 \cdot \frac{1}{2} \cdot \frac{3}{4} \cdot \frac{4}{5} = 0.3 > \frac{1}{4}$ in X . The green rectangle $\mathcal{N}(P')$ contains one point, which corresponds to an occurrence of P' in S_3 . However, this occurrence does not respect the property Π (because $i + |P'| - 1 = 2 + 4 - 1 = 5 > \pi_3[2] = 4$) and therefore is a false positive in X (it occurs with probability $0.15 < \frac{1}{4}$). The orange rectangle $\mathcal{N}(P'')$ does not contain any point, because the pattern does not occur in the z -estimation. ■

IV. SPACE-EFFICIENT CONSTRUCTION OF THE INDEX

Recall that to construct the index in Section III, we first construct a z -estimation, which temporarily takes $\Theta(nz)$ space during construction. In this section, we improve the space required for the construction of the index by designing a space-efficient algorithm for constructing a minimizer solid factor tree with only a moderate increase in the construction time.

Main Idea. We start the index construction by *simulating* the construction of an *extended solid factor tree*. This is a trie of the solid factors of X extended by H_X , the heavy string of X . In particular, we maintain the subtree induced by the solid factors that *start at minimizer positions* but discard the nodes that we do not need upon returning to their parents. We achieve this via traversing the full tree in a DFS order. Thus, even though the full tree size is $\mathcal{O}(nz)$ (Lemma 10), at any moment, we store *only* the current leaf-to-root path plus the actual output. Therefore, we use only $\mathcal{O}(n + \frac{nz}{\ell} \log z)$ expected space at a cost of $\mathcal{O}(nz \log \ell)$ time (Lemma 11). We next reverse this tree (the solid factors are read from leaf to root there, while in the minimizer solid factor tree those are read from root to leaf), in $\mathcal{O}(\frac{nz}{\ell} \log \frac{nz}{\ell} \log z)$ expected time, using $\mathcal{O}(\frac{nz}{\ell} \log z)$ expected space (Theorem 12). *Our approach thus trades construction time for less space:* in total, it adds up to $\mathcal{O}(nz \log \ell + \frac{nz}{\ell} \log \frac{nz}{\ell} \log z)$ expected construction time (instead of $\mathcal{O}(nz)$) but $\mathcal{O}(n + \frac{nz}{\ell} \log z)$ expected construction space (instead of $\mathcal{O}(nz)$). We thus arrive to Contribution 2.

Key Concepts. The string $U \cdot H_X[j+1..n]$ (resp. $(H_X[1..i-1] \cdot U)^r$) is called the *right extension of the solid factor U* (resp. *left extension of the solid factor U*), if U is a solid factor of X starting at position i and ending at position $j \geq i - 1$.

For such a U , we define a *forward extended solid factor tree* of X as a trie of all the reversals of $U \cdot H_X[j+1..n]$, and a *backward extended solid factor tree* of X as a trie of all $H_X[1..i-1] \cdot U$. These trees can be constructed by looking only at the extensions of *maximal* solid factors, namely those that cannot be extended into a longer solid factor [18]. We employ the following lemma to bound the size of the trees:

Lemma 10 ([18]). *The extended solid factor trees have $\mathcal{O}(nz)$ nodes.*

We next describe our algorithm (see [37] for pseudocode). **Construction.** We start by constructing the minimizer versions of the extended solid factor trees – that is for the solid factors trimmed to their parts starting (resp. ending) at the position of their minimizer. In particular, we show how to construct the

forward extended solid factor tree (see Fig. 4) – the backward one can be constructed by simply doing the same operations on the reversed string, except that the minimizers will be computed on the reversed substrings.

Initialization. We construct the tree with a DFS traversal of the full (non-minimizer) extended solid factor tree, starting from the root, which corresponds to ε , the empty string. Each node corresponds to the right extension of a solid factor U of X starting at a position i (recall that U can be empty, in which case its right extension is $H_X[i..n]$).

First Visit to a Node. When a node u that corresponds to a solid factor U starting at position i of X and ending at position j is created, we keep a pair of labels (i, Diff) , where Diff is the sequence (list) of differences (mismatches) between U and $H_X[i..j]$. By Lemma 3, the label of a given node has size $\mathcal{O}(\log z)$. Note also that, for any ancestor of a node u , its list of mismatches will be a suffix of Diff .

A single node and equivalently such a pair of labels can still represent multiple solid factors (for different values of j – if the suffix of the solid factor matches the heavy string); henceforth, by U we mean the shortest such solid factor: j is the largest element of Diff or $j = i - 1$ if Diff is empty.

Additionally, given a node u representing $S = U \cdot H_X[j + 1..n]$, we check if the longest solid prefix of S has length at least ℓ ; we check this in $\mathcal{O}(1)$ time using value p – a global variable that denotes the probability of the current node – that is the weight of U and the precomputed array PP_H of prefix products of H_X for the heavy part. If this is the case, we ask for the minimizer μ of this solid factor, and mark the $(\mu - 1)$ th ancestor of u as a minimizer node. Such minimizer can be found in $\mathcal{O}(1)$ time using a heap data structure [30], which stores information about the length- k substrings of the length- ℓ prefix of $S = U \cdot H_X[j + 1..n]$ and is updated in $\mathcal{O}(\log \ell)$ time in each step of the traversal.

Stepping Down to a Child Node. If U is empty, then the node u corresponds to a string $H_X[i..n]$. In this case, p is not updated when creating its child v corresponding to $H_X[i - 1..n]$. This way, we ensure by induction that $p = 1$ at the creation of each such node, and only for such a node, so that this can be checked in $\mathcal{O}(1)$ time. We now assume that the created child does not correspond to $H_X[i - 1..n]$. To create a child v of the node u , corresponding to the right extension of the string $\alpha \cdot U$ for some letter $\alpha \in \Sigma$, one needs to check if $\alpha \cdot U$ is valid by computing its probability (U is nonempty, or the letter α is different from $H_X[i - 1]$). This is done using p , which we multiply by $p_{i-1}[\alpha]$. In any case, the labels of v are computed from the labels of u by decreasing the starting position and potentially adding a new label to Diff .

Returning to the Parent Node. In the DFS we traverse the full extended solid factor tree, but we are only interested in the strings that start in those minimizer nodes. Thus, we remove the nodes which correspond to letters of the solid factors that appear before the position of the first minimizer and recursively compactify the tree during the traversal.

After all descendants of u are created, we keep u explicit if it is a minimizer node or if it has more than one (not removed)

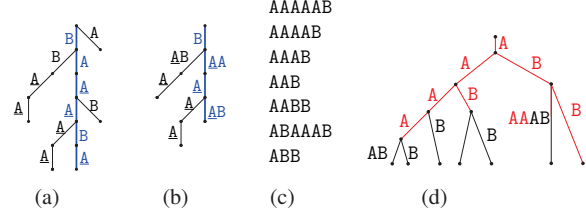


Fig. 4: (a) The forward extended solid factor tree with X from Example 1. The blue edges correspond to the heavy string $H_X = ABAAAAB$ (reversed). The minimizer positions are underlined. (b) The minimizer extended solid factor tree. The edges without any minimizer descendant nodes are pruned and the non-minimizer nodes are made implicit. (c) The lexicographically sorted strings corresponding to each path from a minimizer node to a root. (d) The minimizer solid factor tree constructed by Theorem 12. It contains the forward (top) tree from Fig. 3 as a red subtree. The edge with no label is added in the figure to stress that AB also has a corresponding leaf. In the algorithm, we simply make the corresponding internal node explicit and treat it as a leaf node.

child. Otherwise, the node u is made implicit by merging it with its parent. Finally, upon returning to the parent of u we update p by dividing it by $p_i[U[1]]$ (if $p < 1$) and the list of differences by removing position i if $i \in \text{Diff}$.

Complexity Analysis. By Lemma 5 the final tree has $\mathcal{O}(\frac{nz}{\ell})$ nodes in expectation. As for the construction space, observe that while a node u is being processed, only the path between u and the root is uncompactified, and contains at most n nodes. All the other global variables also have size $\mathcal{O}(n)$, therefore the total expected work space needed is $\mathcal{O}(\frac{nz}{\ell} \log z + n)$. As for the construction time, note that the set of created nodes is exactly the set of nodes from the original extended solid factor tree (namely, without minimizers), which has size $\mathcal{O}(nz)$ by Lemma 10. Reading X and creating H_X costs $n\sigma = \mathcal{O}(n)$ time [24]; after that the total number of probabilities read is bounded by the number of tree nodes, as for a single position we can read those in the order of non-increasing probabilities.

During the construction, all the operations cost $\mathcal{O}(1)$ time with the exception of updating the minimizer heap which takes $\mathcal{O}(\log \ell)$ time and storing a copy of the list of differences for each minimizer node which takes $\mathcal{O}(\log z)$ time. Note that the last type of the operation does not influence the worst-case running time as we can abandon the computation upon learning that the total size of those lists reaches nz – in which case the classic (non-minimizer) data structure is more efficient. We have thus proved the following lemma.

Lemma 11. For any weighted string of length n , any weight threshold $\frac{1}{z}$, and any integer $\ell > 0$, we can construct a representation of the minimizer extended solid factor trees in $\mathcal{O}(nz \log \ell)$ time using $\mathcal{O}(n + \frac{nz \log z}{\ell})$ expected space.

Main Result. The minimizer solid factors tree can be constructed from the minimizer extended solid factor tree in

$\mathcal{O}(\frac{nz}{\ell} \log \frac{nz}{\ell} \log z)$ expected time and $\mathcal{O}(\frac{nz}{\ell} \log z)$ expected space (the details of the construction can be found in [37]), hence we arrive at the main result of the section:

Theorem 12. *For any weighted string X of length n , any weight threshold $\frac{1}{z}$, and any integer $\ell > 0$, we can construct the minimizer solid factor tree in expected time $\mathcal{O}(nz \log \ell + \frac{nz}{\ell} \log \frac{nz}{\ell} \log z)$ and space $\mathcal{O}(n + \frac{nz}{\ell} \log z)$.*

V. PRACTICALLY FAST QUERYING WITHOUT A GRID

In this section, we describe a simple and fast querying algorithm that does not require the grid to be constructed on top of the trees. This querying algorithm has worse guarantee than that in Theorem 9 but performs much better in practice, due to its simplicity, as we show in the experimental evaluation.

Like in the previous construction let $\mu = f(P[1..\ell])$. Without loss of generality we assume that $\mu \leq \frac{m}{2}$ (otherwise we swap the roles of the parts of P and of trees $\mathcal{T}_{\text{suff}}$ and $\mathcal{T}_{\text{pref}}$). Let u be the node reached by reading $P[\mu..m]$ in $\mathcal{T}_{\text{suff}}$. We can separately check each leaf in the subtree of u as a potential candidate in $\mathcal{O}(m)$ time: we can do this assuming we have random access to X . This time we cannot use the $\frac{nz}{\sigma^m}$ bound on the expected number of candidates from Lemma 8. However, $P[\mu..m]$ has length at least $m/2$, and hence the expected number of candidates can still be bounded by $\sum_{k=\lceil m/2 \rceil}^m \frac{nz}{\sigma^k} \leq 2 \frac{nz}{\sigma^{m/2}}$ using a similar argument. Thus we can answer a query in $\mathcal{O}(m \cdot (1 + \frac{nz}{\sigma^{m/2}}))$ expected time.

Example 8. For the weighted string from Example 1 and pattern $P' = \text{B}\underline{\text{A}}\text{AB}$ the grid based construction finds a single candidate (3, 3) to check (see Fig. 3 and Example 7). In case of the simpler querying algorithm we only locate one part of the pattern in one of the trees, in this case $|\text{B}\underline{\text{A}}| < |\underline{\text{A}}\text{AB}|$, hence we locate AAB in $\mathcal{T}_{\text{suff}}$. We find two leaves with labels (3, 3) and (4, 2), that is the candidate occurrences starting at positions $2 = 3 - |\text{B}\underline{\text{A}}| + 1$ and $3 = 4 - |\text{B}\underline{\text{A}}| + 1$. Each of those can be checked naively to get the occurrence probabilities $3/20$ and $3/40$ respectively; each probability is smaller than $1/4$. ■

VI. RELATED WORK

The *Weighted Indexing* problem was introduced by the work of Iliopoulos et al. [46]; the authors gave a tree-based index supporting $\mathcal{O}(m + |\text{Occ}|)$ -time queries. The construction time, space, and size of the index was, however, $\mathcal{O}(n\sigma^z \log z)$. Their index is essentially a compacted trie of all the solid factors of X . Amir et al. [13] then reduced the Weighted Indexing problem to the so-called *Property Indexing* problem in a standard string of length $\mathcal{O}(nz^2 \log z)$. For the latter problem, Amir et al. proposed a super-linear-time construction and $\mathcal{O}(m + |\text{Occ}|)$ -time queries. Later, it was shown that Property Indexing can be performed in linear time [17]. This directly gives a solution to the Weighted Indexing problem with construction time, space, and size $\mathcal{O}(nz^2 \log z)$, preserving the optimal query time. The state-of-the-art indexes for Weighted Indexing are the weighted suffix tree (WST) [17], [18] and the weighted suffix array (WSA) [23]; see the Introduction for more details. To conclude, *there are currently no practical*

TABLE II: Characteristics of the real datasets we used.

Dataset	# of samples	Length n	Δ as percentage of n	Size of z -estimation for the default z (MBs)
SARS	1,181	29,903	3.6%	31
EFM	1,432	2,955,294	6%	378
HUMAN	2,504	35,194,566	3.2%	282
RSSI	N/A	6,053,462	100%	96.9

indexing schemes for Weighted Indexing mainly due to their prohibitive size and space requirements.

There is substantial work on practical indexing schemes for probabilistic/uncertain data; e.g., for range queries [78], [25], [79], [28], [8], top- k queries [88], [45], [89], [71], nearest neighbor queries [7], [27], [26], sql-like queries [70], inference queries [51], and probabilistic equality threshold queries [75]. These indexes were developed for different uncertainty data models, such as tuple uncertainty and attribute uncertainty [76]. Under tuple uncertainty, the presence of a tuple in a relation is probabilistic, while under attribute uncertainty a tuple is certainly present in a database but one or more of its attributes are not known with certainty. Several indexes are built on R-trees or inverted indexes (e.g., [75], [32]), while others are built on R*-trees [78]. There are also specialized indexes, e.g., for probabilistic XML queries [40] or uncertain graphs [87], [77]. Our work differs substantially from the above in the supported data model and query type.

VII. EXPERIMENTAL EVALUATION

A. Data and Setup

Data. We used three real weighted strings which model variations found in the DNA ($\sigma = 4$) of different samples of the same species. The chromosomal or genomic location of a gene or any other genetic element is called a *locus* and alternative DNA sequences at a locus are called *alleles*. Allele frequency, or gene frequency, is the relative frequency of an allele at a particular locus in a population, expressed as a fraction or percentage. Thus, alleles have a natural representation as weighted strings: we model the probability $p_i(\alpha)$ in these strings as the relative frequency of letter α at position i among the different samples.

We next describe the datasets we used (see also Table II):

- SARS: The full genome of *SARS-CoV-2* (isolate Wuhan-Hu-1) [1] combined with a set of single nucleotide polymorphisms (SNPs) [2] taken from 1,181 samples [81].
- EFM: The full chromosome of *Enterococcus faecium* Aus0004 strain (CP003351) [3] combined with a set of SNPs [4] taken from 1,432 samples [29].
- HUMAN: The full chromosome 22 of the *Homo sapiens* genome (v. GRCh37) [5] combined with a set of SNPs [6] taken from the final phase of the 1000 Genomes Project (phase 3) representing 2,504 samples [34].

The percentage of positions where more than one letter has a probability of occurrence larger than 0 is denoted by Δ .

We also used another real weighted string comprised of Received Signal Strength Indicator (RSSI) values of sensors; $p_i(\alpha)$, $i \in [1, n]$, corresponds to the ratio of IEEE 802.15.4

channels that received an RSSI value α at time i , and $\sigma = 91$ [68]. The characteristics of this dataset, called RSSI, are in Table II. Furthermore, we used a family of weighted strings, generated from RSSI. Each such string is denoted by $\text{RSSI}_{n,\sigma}$, where $n \in \{2, 4, 6, 8\}$ is the number of times the length of this string is larger than that of RSSI and $\sigma \in \{16, 32, 64\}$ is its alphabet size. To increase n , we appended RSSI to itself. To reduce σ , we replaced each RSSI value (integer) v with $v \bmod y$, where $y \in \{16, 32, 64\}$ is the desired alphabet size. For all these datasets, $\Delta = 100\%$.

Parameters. For every weighted string of length n , every pattern length $m \in \{64, 128, 256, 512, 1024\}$, and every z we used, we selected $\lfloor nz/200 \rfloor$ patterns from the z -estimation of the weighted string, uniformly at random, to account for the different n and z values. For example, for HUMAN whose length is $n = 35,194,566$, and for $z = 32$, we have selected 5,631,130 patterns uniformly at random from its 32-estimation. The default z for SARS, EFM, HUMAN, RSSI, and $\text{RSSI}_{n,\sigma}$ was 1024, 128, 8, 16, and 16, and led to z -estimations with sizes of several MBs; see Table II. The parameter ℓ was set to m , and the default m value was 256.

Implementations. We used the implementations of the state-of-the-art indexes WST and WSA from [17] and [23], respectively. We implemented: (1) MWST-G, the algorithm underlying our Theorem 9. (2) MWST, a simplified version of MWST-G that drops the 2D grid and performs pattern matching as described in Section V. (3) MWSA and MWSA-G, the *array-based* versions of MWST and MWST-G, respectively. Note that an in-order DFS traversal of the tree gives the array. (4) MWST-SE, the space-efficient construction of MWST underlying Theorem 12. In all implementations, we used Karp-Rabin fingerprints [53] to compute the minimizers.

Measures. We used all four relevant measures of efficiency (see Introduction): index size; query time; construction space; and construction time. To measure the query and construction time, we used the `chrono` C++ library. To measure the index size, we used the `malloc2` C++ function. To measure the construction space, we recorded the maximum resident set size using the `/usr/bin/time -v` command.

Environment. All experiments ran using a single AMD EPYC 7282 CPU at 2.8GHz with 252GB RAM under GNU/Linux. All methods were implemented in C++ and compiled with g++ (v. 12.2.1) at optimization level `-O3`.

Code and Datasets. The code and all datasets are available at <https://github.com/solonas13/ius> under GNU GPL v3.0.

B. Evaluating our Minimizer-based Indexes

Index Size. Figs. 5 and 6 show that our tree-based (resp. array-based) indexes occupy *up to two orders of magnitude less space* than WST (resp. WSA). The size of our indexes decreases with ℓ and increases with z (see Theorem 9). Furthermore, the array-based indexes occupy several times less space than the tree-based ones, as it is widely known [52]. For example, note from Figs. 5c and 5d that for $\ell = 1024$, WST occupied 126GB of space, whereas our MWST 900MB and MWSA only 204MB! As expected, our grid-based indexes

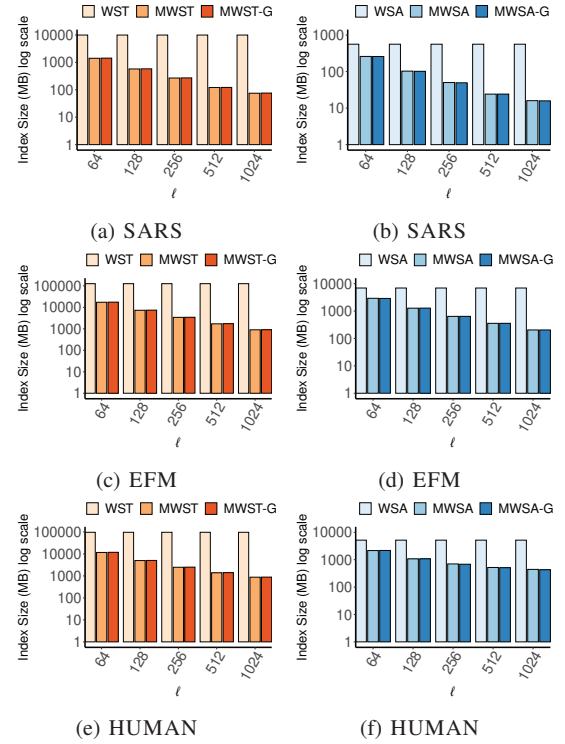


Fig. 5: Index size (log scale, MB) vs. ℓ .

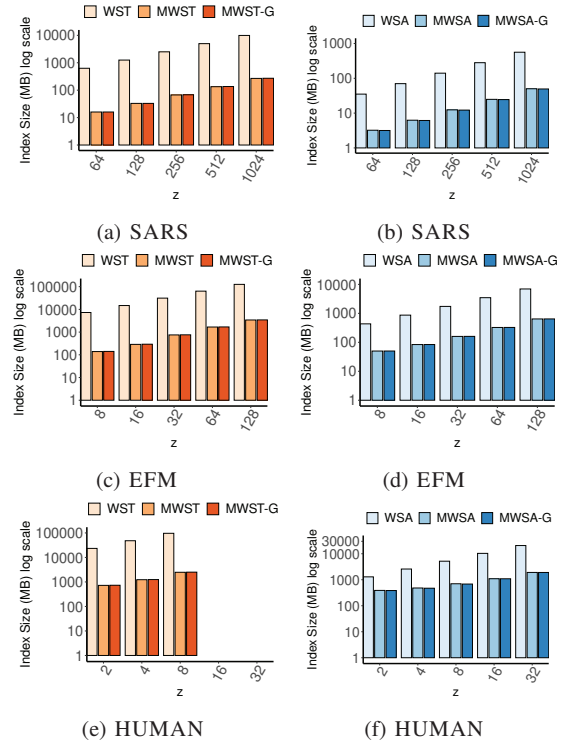


Fig. 6: Index size (log scale, MB) vs. z . The tree-based indexes for HUMAN (Fig. 6e) needed $> 252\text{GB}$ of space when $z \geq 16$ and hence could not be constructed.

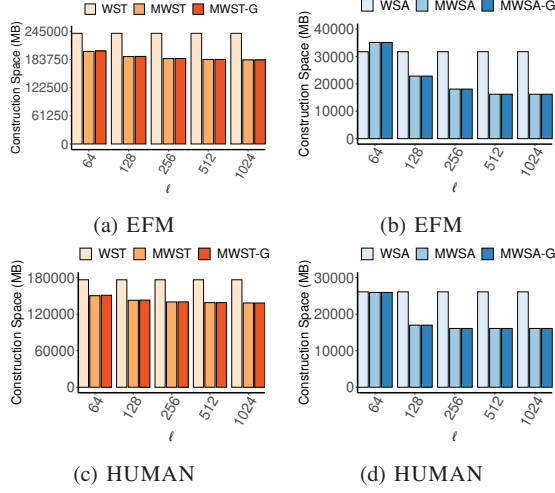


Fig. 7: Construction space (MB) vs. ℓ .

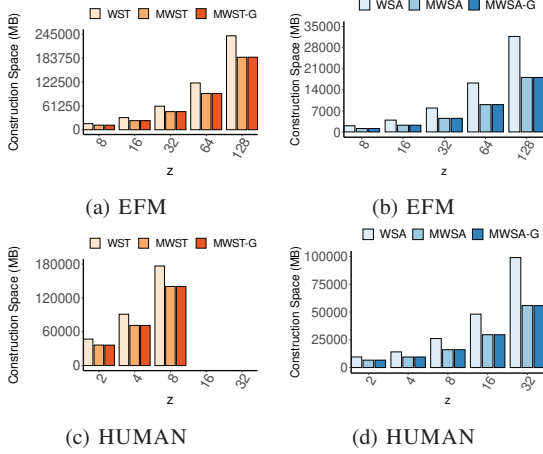


Fig. 8: Construction space (MB) vs. z . The tree-based indexes for HUMAN (Fig. 8c) needed $> 252\text{GB}$ when $z \geq 16$ and hence could not be constructed.

MWST-G and MWSA-G occupy a slight amount of extra space compared to MWST and MWSA, respectively.

Construction Space. Figs. 7 and 8 show that our tree-based (resp. array-based) indexes outperform WST (resp. WSA) by 27% (resp. 61%) on average; the results for SARS are analogous to those for EFM and are omitted for space. Although our construction algorithm (see Theorem 9) takes $\Theta(nz)$ space in any case, it carries lower constant factors than that of WST. That is, in practice, the index construction space for our tree-based indexes decreases as ℓ increases and increases with z – see Lemmas 5 and 7, which show a clear dependency on the number $\mathcal{O}(\frac{nz}{\ell})$ of sampled minimizers. The same explanation holds for WSA and our array-based indexes. Again, as it is widely known [52], the array-based indexes outperform the tree-based ones in terms of space; and, as expected, MWST-G and MWSA-G need a very slightly larger

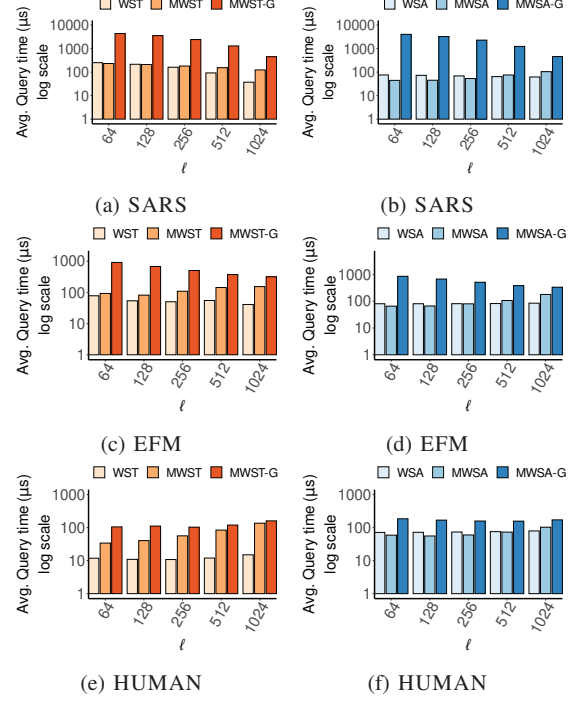


Fig. 9: Average query time (log scale, μs) vs. ℓ .

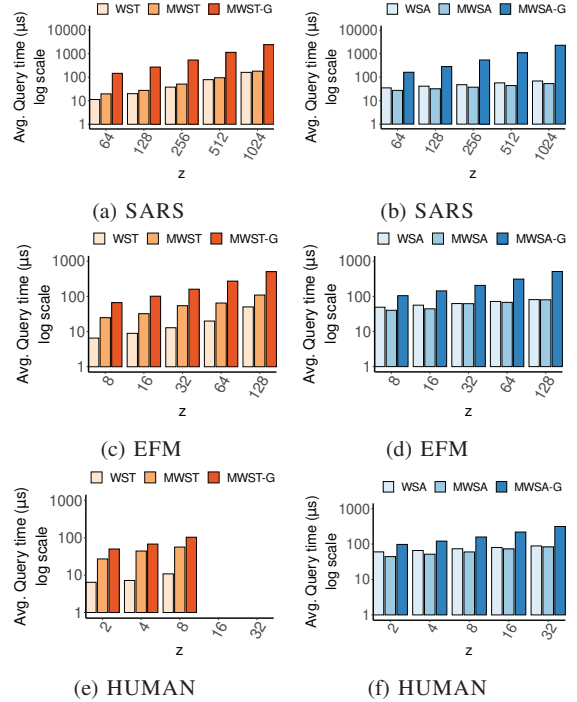


Fig. 10: Average query time (log scale, μs) vs. z . The tree-based indexes for HUMAN (Fig. 10e) needed $> 252\text{GB}$ when $z \geq 16$ and hence could not be constructed.

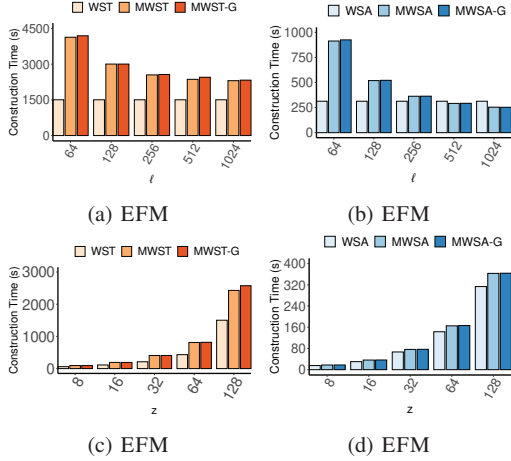


Fig. 11: (a, b) Construction time (s) vs. ℓ for EFM. (c, d) Construction time (s) vs. z for EFM. The results for SARS and HUMAN were analogous (omitted for space).

construction space than MWST and MWSA, respectively.

Query Time. Figs. 9 and 10 show that MWST is generally slower than WST because its search operation is more costly than that of WST (see Theorem 9). However, MWSA is competitive to WSA since, due to the smaller size of the former, the binary search operation used in query answering (pattern matching) [64] becomes faster. This is *very encouraging* given its substantially smaller index size and index construction space across all z and ℓ values. Furthermore, MWST and MWSA outperform MWST-G and MWSA-G, respectively. This is in line with the findings of [16], [62], which show that simple verification schemes like the one developed by us in Section V, are faster than grid approaches, even if the theoretical guarantees provided by the former are weaker. Note in Fig. 9 that the query time of the grid-based indexes is not negatively affected by increasing ℓ , unlike MWST and MWSA. In particular, Fig. 9 shows that, although the grid-based indexes are slower, the difference in performance *decreases* as ℓ grows. This is because, as ℓ grows, the grid becomes smaller and the simple verification schemes become more expensive, which highlights the benefit of Theorem 9. The query time of all indexes increases with z , as expected by their time complexities. The query time of WST and WSA does not depend on ℓ , as expected by their time complexities.

Construction Time. Fig. 11 shows that WST and WSA can be constructed in less time than our tree-based and array-based indexes, respectively. This is expected as our construction is much more complex than that of WST and WSA [17], [23]. In particular, although our construction algorithm (see Theorem 9) takes $\Theta(nz)$ time in any case, it carries higher constant factors than those of WST and WSA. This is expected as, in some sense, our construction largely follows the one of WST and WSA but it does additional work implied by the sampling mechanism. In practice, the construction time decreases as ℓ increases and increases with z – see Lemmas 5

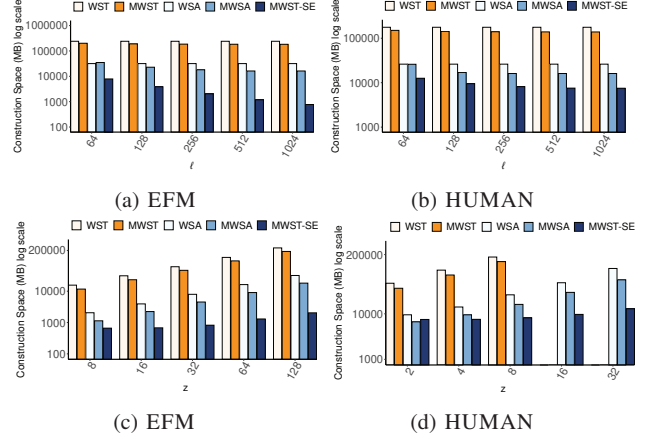


Fig. 12: Construction space (log scale, MB) vs: (a, b) ℓ . (c, d) z . WST and MWST for HUMAN (Fig. 12d) needed $> 252\text{GB}$ when $z \geq 16$ and hence could not be constructed. The results for SARS were analogous (omitted for space).

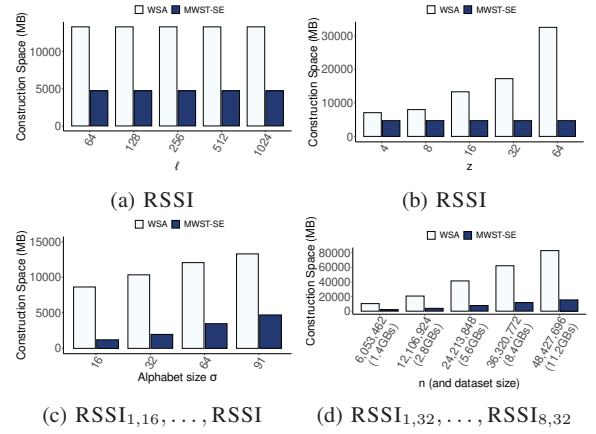


Fig. 13: Construction space (MBs) vs. (a) ℓ , (b) z , (c) σ , and (d) n (and dataset size).

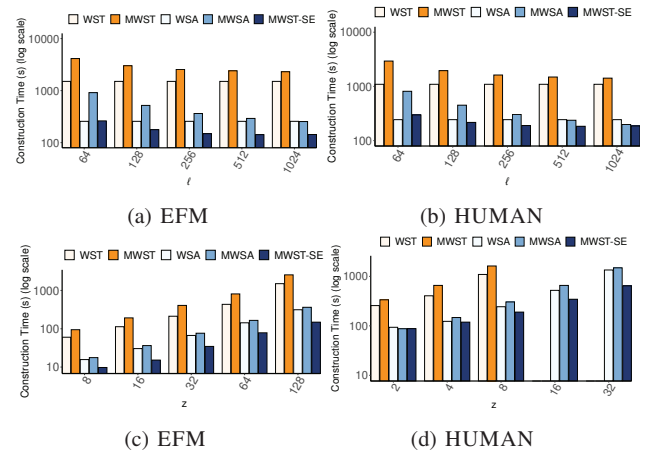


Fig. 14: Construction time (log scale, s) vs: (a, b) ℓ . (c, d) z . WST and MWST for HUMAN (Fig. 14d) needed $> 252\text{GB}$ when $z \geq 16$ and hence could not be constructed. The results for SARS were analogous (omitted for space).

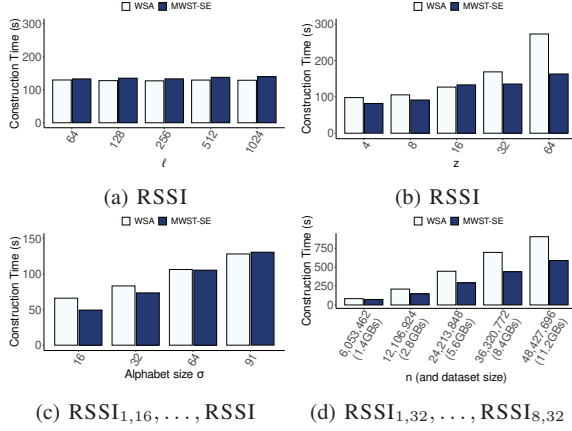


Fig. 15: Construction time (s) vs. (a) ℓ , (b) z , (c) σ , and (d) n (and dataset size).

and 7, which show a clear dependency on the number $\mathcal{O}(\frac{nz}{\ell})$ of sampled minimizers. On average, MWST requires 70% (resp. MWSA requires 32%) more time to be constructed than WST (resp. WSA). MWST-G and MWSA-G have similar construction time to MWST and MWSA, respectively.

C. Evaluating our Space-efficient Index Construction

Construction Space. Fig. 12 shows that the construction space of MWST-SE is up to *one order of magnitude smaller* than that of WSA and *52 times smaller* than that of MWST on average. The construction space of MWST-SE decreases with ℓ and increases with z , as expected by Theorem 12. For example, in Fig. 12a MWST-SE needs only 772MB of memory to be constructed when $\ell = 1024$, while WSA and MWST need over 32GBs and 183GBs, respectively. Even for $\ell = 64$, the construction space of MWST-SE is 4 times smaller than that of WSA and more than 25 times smaller than that of MWST. On the RSSI datasets, MWST-SE substantially outperformed all other indexes. Fig. 13 shows the results against the best competitor, WSA. MWST-SE needed 4 times less space than WSA on average. Although z and ℓ appear in the space bound of Theorem 12, and σ does not, the impact of the latter is larger on the construction space. The reason is that the maximum resident set size we measure includes the memory for reading the dataset file, which increases with σ for all indexes. Note that the construction space for both indexes scaled linearly with n , as can be seen in Fig. 13d.

Construction Time. Fig. 14 shows that the construction time of MWST-SE is on average 53% smaller than that of WSA, the next fastest index. This is *very encouraging*, as MWST-SE is quite complex. The construction time of MWST-SE decreases with ℓ and increases with z , as expected by Theorem 12. For example, for $\ell = 1024$ and $z = 128$ in Fig. 14a, the construction time of MWST-SE is smaller by 31% (resp. 16 times smaller) compared to that of WSA (resp. MWST). This faster construction is a consequence of WSA and WST being always of $\Theta(nz)$ size (producing copies

of solid factors), while in the extended solid factor trees each solid factor is considered only once. The extra $\mathcal{O}(\log \ell)$ cost for heap operations is very optimized and in practice comparable with the large constants of the other constructions for reasonable ℓ values.

On the RSSI datasets, MWST-SE substantially outperformed all other indexes. Fig. 15 shows the results against the best competitor, WSA. Now the term $n\sigma = \mathcal{O}(n)$ prevails in the construction time bound of Theorem 12. Thus, the construction time increased linearly with σ (Fig. 15c), while the impact of ℓ and z was smaller. MWST-SE scaled linearly with n (Fig. 15d). The input datasets we used are in the order of GBs, thus their reading takes much of the construction time.

VIII. DISCUSSION: LIMITATIONS AND FUTURE WORK

The size of our indexes is $\mathcal{O}(\frac{nz}{\ell} \log z)$ in expectation because minimizers have no worst-case guarantees; e.g., in string *abcdefg...*, every position is a minimizer. This could be avoided by using *difference covers* [52] instead. Difference covers are practical to construct, but slash the set of positions by $\sqrt{\ell}$ instead of ℓ (the factor that minimizers slash by in real-world datasets). As our focus is on practical schemes for reducing the space, we have resorted to minimizers. There are other sampling schemes slashing the set of positions by ℓ in the worst case [55], such as *string synchronizing sets*; these are however impractical to construct (see [33], for example) and not directly applicable to the uncertain setting.

Our methods do not explicitly take advantage of the n probability distributions. The following may be explored to improve our indexes: (i) use the percentage Δ of positions where more than one letter has a probability of occurrence larger than 0 as a complexity parameter; (ii) use lower or upper bounds on the letter probabilities depending on the underlying application. For instance, in sequencing data, a quality score is assigned to a single nucleotide in every position i , which is then translated to a probability p . The other 3 nucleotides can be assigned to probability $(1 - p)/3$. However, based on the vicinity of the i th position or on domain knowledge, we could rather use upper or lower bounds for the remaining letters.

Our indexes rely explicitly or implicitly on the z -estimation of the input weighted string. This is a family $\mathcal{S}$ of $[z]$ strings of length n . Although the construction of $\mathcal{S}$ is combinatorially correct [17] it is oblivious to domain knowledge. It may thus generate solid factors that are completely impossible to occur in a real-world dataset. For instance, in biology, such implausible factors are termed *absent* or *avoided* [12]. If one had this knowledge, they could easily trim these factors from the index by first querying them during a post-processing step. It seems much more challenging though to amend our data model to include this knowledge while constructing the index.

ACKNOWLEDGMENTS

This work was supported by the projects that have received funding from the European Union's Horizon 2020 research and innovation programme under the Marie Skłodowska-Curie grant agreements No 872539, No 956229, and No 101034253.

REFERENCES

- [1] <https://www.ncbi.nlm.nih.gov/nuccore/MN908947.3>.
- [2] <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8363274/bin/elife-66857-sup2.txt>.
- [3] <https://www.ncbi.nlm.nih.gov/nuccore/CP003351>.
- [4] https://github.com/francescoll/powerbagwas/blob/main/data/efm_clade_all.vcf.gz.
- [5] https://www.ncbi.nlm.nih.gov/datasets/genome/GCF_000001405.13/.
- [6] https://ftp.1000genomes.ebi.ac.uk/vol1/ftp/release/20130502/ALL.chr22.phase3_shapeit2_mvncall_integrated_v5b.20130502.genotypes.vcf.gz.
- [7] Pankaj K. Agarwal, Boris Aronov, Sarel Har-Peled, Jeff M. Phillips, Ke Yi, and Wuzhou Zhang. Nearest-neighbor searching under uncertainty II. *ACM Trans. Algorithms*, 13(1):3:1–3:25, 2016.
- [8] Pankaj K. Agarwal, Siu-Wing Cheng, Yufei Tao, and Ke Yi. Indexing uncertain data. In Jan Paredaens and Jianwen Su, editors, *Proceedings of the Twenty-Eighth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, PODS 2009, June 19 - July 1, 2009, Providence, Rhode Island, USA*, pages 137–146. ACM, 2009.
- [9] Charu C. Aggarwal. On unifying privacy and uncertain data models. In *Proceedings of the 24th International Conference on Data Engineering (ICDE)*, pages 386–395. IEEE Computer Society, 2008.
- [10] Charu C. Aggarwal. *Managing and Mining Uncertain Data*, volume 35 of *Advances in Database Systems*. Kluwer, 2009.
- [11] Charu C. Aggarwal, editor. *Managing and Mining Sensor Data*. Springer, 2013.
- [12] Yannis Almirantis, Panagiotis Charalampopoulos, Jia Gao, Costas S. Iliopoulos, Manal Mohamed, Solon P. Pissis, and Dimitris Polychronopoulos. On avoided words, absent words, and their application to biological sequence analysis. *Algorithms Mol. Biol.*, 12(1):5:1–5:12, 2017.
- [13] Amihoud Amir, Eran Chencinski, Costas S. Iliopoulos, Tsvi Kopelowitz, and Hui Zhang. Property matching and weighted matching. In Moshe Lewenstein and Gabriel Valiente, editors, *Combinatorial Pattern Matching, 17th Annual Symposium, CPM 2006, Barcelona, Spain, July 5-7, 2006, Proceedings*, volume 4009 of *Lecture Notes in Computer Science*, pages 188–199. Springer, 2006.
- [14] Amihoud Amir, Eran Chencinski, Costas S. Iliopoulos, Tsvi Kopelowitz, and Hui Zhang. Property matching and weighted matching. *Theor. Comput. Sci.*, 395(2-3):298–310, 2008.
- [15] Mozdeh Arianneshad, Ali Montazerlghaem, Hamed Zamani, and Azadeh Shakery. Improving retrieval performance for verbose queries via axiomatic analysis of term discrimination heuristic. In Noriko Kando, Tetsuya Sakai, Hideo Joho, Hang Li, Arjen P. de Vries, and Ryen W. White, editors, *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval, Shinjuku, Tokyo, Japan, August 7-11, 2017*, pages 1201–1204. ACM, 2017.
- [16] Lorraine A. K. Ayad, Grigorios Loukides, and Solon P. Pissis. Text indexing for long patterns: Anchors are all you need. *Proc. VLDB Endow.*, 16(9):2117–2131, 2023.
- [17] Carl Barton, Tomasz Kociumaka, Chang Liu, Solon P. Pissis, and Jakub Radoszewski. Indexing weighted sequences: Neat and efficient. *Inf. Comput.*, 270, 2020.
- [18] Carl Barton, Tomasz Kociumaka, Solon P. Pissis, and Jakub Radoszewski. Efficient index for weighted sequences. In Roberto Grossi and Moshe Lewenstein, editors, *27th Annual Symposium on Combinatorial Pattern Matching, CPM 2016, June 27-29, 2016, Tel Aviv, Israel*, volume 54 of *LIPIcs*, pages 4:1–4:13. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016.
- [19] Michael Bendersky and W. Bruce Croft. Discovering key concepts in verbose queries. In Sung-Hyon Myaeng, Douglas W. Oard, Fabrizio Sebastiani, Tat-Seng Chua, and Mun-Kew Leong, editors, *Proceedings of the 31st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2008, Singapore, July 20-24, 2008*, pages 491–498. ACM, 2008.
- [20] Sudip Biswas, Manish Patil, Sharma V. Thankachan, and Rahul Shah. Probabilistic threshold indexing for uncertain strings. In Evaggelia Pitoura, Sofian Maabout, Georgia Koutrika, Amélie Marian, Letizia Tanca, Ioana Manolescu, and Kostas Stefanidis, editors, *Proceedings of the 19th International Conference on Extending Database Technology, EDBT 2016, Bordeaux, France, March 15-16, 2016, Bordeaux, France, March 15-16, 2016*, pages 401–412. OpenProceedings.org, 2016.
- [21] Peter A. Boncz, Thomas Neumann, and Viktor Leis. FSST: fast random access string compression. *Proc. VLDB Endow.*, 13(11):2649–2661, 2020.
- [22] Timothy M. Chan, Kasper Green Larsen, and Mihai Pătrăscu. Orthogonal range searching on the RAM, revisited. In Ferran Hurtado and Marc J. van Kreveld, editors, *Proceedings of the 27th ACM Symposium on Computational Geometry, Paris, France, June 13-15, 2011*, pages 1–10. ACM, 2011.
- [23] Panagiotis Charalampopoulos, Costas S. Iliopoulos, Chang Liu, and Solon P. Pissis. Property suffix array with applications in indexing weighted sequences. *ACM J. Exp. Algorithmics*, 25:1–16, 2020.
- [24] Panagiotis Charalampopoulos, Costas S. Iliopoulos, Solon P. Pissis, and Jakub Radoszewski. On-line weighted pattern matching. *Inf. Comput.*, 266:49–59, 2019.
- [25] Lu Chen, Yunjun Gao, Aoxiao Zhong, Christian S. Jensen, Gang Chen, and Baihua Zheng. Indexing metric uncertain data for range queries and range joins. *VLDB J.*, 26(4):585–610, 2017.
- [26] Reynold Cheng, Lei Chen, Jinchuan Chen, and Xike Xie. Evaluating probability threshold k-nearest-neighbor queries over uncertain data. In Martin L. Kersten, Boris Novikov, Jens Teubner, Vladimir Polutin, and Stefan Manegold, editors, *EDBT 2009, 12th International Conference on Extending Database Technology, Saint Petersburg, Russia, March 24-26, 2009, Proceedings*, volume 360 of *ACM International Conference Proceeding Series*, pages 672–683. ACM, 2009.
- [27] Reynold Cheng, Dmitri V. Kalashnikov, and Sunil Prabhakar. Querying imprecise data in moving object environments. *IEEE Trans. Knowl. Data Eng.*, 16(9):1112–1127, 2004.
- [28] Reynold Cheng, Yuni Xia, Sunil Prabhakar, Rahul Shah, and Jeffrey Scott Vitter. Efficient indexing methods for probabilistic threshold queries over uncertain data. In Mario A. Nascimento, M. Tamer Özsu, Donald Kossmann, Renée J. Miller, José A. Blakeley, and K. Bernhard Schiefer, editors, *(e)Proceedings of the Thirtieth International Conference on Very Large Data Bases, VLDB 2004, Toronto, Canada, August 31 - September 3 2004*, pages 876–887. Morgan Kaufmann, 2004.
- [29] Francesc Coll, Theodore Goulouris, Sebastian Bruchmann, Jody Phelan, Kathy E. Raven, Taane G. Clark, Julian Parkhill, and Sharon J. Peacock. PowerBacGWAS: a computational pipeline to perform power calculations for bacterial genome-wide association studies. *Communications Biology*, 5(266), 2022.
- [30] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, 3rd Edition*. MIT Press, 2009.
- [31] Maxime Crochemore, Christophe Hancart, and Thierry Lecroq. *Algorithms on strings*. Cambridge University Press, 2007.
- [32] Xiangyuan Dai, Man Lung Yiu, Nikos Mamoulis, Yufei Tao, and Michail Vaitis. Probabilistic spatial queries on existentially uncertain data. In Claudia Bauzer Medeiros, Max J. Egenhofer, and Elisa Bertino, editors, *Advances in Spatial and Temporal Databases, 9th International Symposium, SSTD 2005, Angra dos Reis, Brazil, August 22-24, 2005, Proceedings*, volume 3633 of *Lecture Notes in Computer Science*, pages 400–417. Springer, 2005.
- [33] Patrick Dinklage, Johannes Fischer, Alexander Herlez, Tomasz Kociumaka, and Florian Kurpicz. Practical performance of space efficient data structures for longest common extensions. In Fabrizio Grandoni, Grzegorz Herman, and Peter Sanders, editors, *28th Annual European Symposium on Algorithms, ESA 2020, September 7-9, 2020, Pisa, Italy (Virtual Conference)*, volume 173 of *LIPIcs*, pages 39:1–39:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.
- [34] Susan Fairley, Ernesto Lowy-Gallego, Emily Perry, and Paul Flicek. The International Genome Sample Resource (IGSR) collection of open human genomic variation resources. *Nucleic Acids Research*, 48(D1):D941–D947, 10 2019.
- [35] Martin Farach. Optimal suffix tree construction with large alphabets. In *38th Annual Symposium on Foundations of Computer Science, FOCS '97, Miami Beach, Florida, USA, October 19-22, 1997*, pages 137–143, 1997.
- [36] Paolo Ferragina and Giovanni Manzini. Indexing compressed text. *J. ACM*, 52(4):552–581, 2005.
- [37] Esteban Gabory, Chang Liu, Grigorios Loukides, Solon P. Pissis, and Wiktor Zuba. Space-efficient indexes for uncertain strings. *CoRR*, abs/2403.14256, 2024.
- [38] Travis Gagie, Gonzalo Navarro, and Nicola Prezza. Fully functional suffix trees and optimal text searching in BWT-runs bounded space. *J. ACM*, 67(1):2:1–2:54, 2020.

- [39] Tingjian Ge and Zheng Li. Approximate substring matching over uncertain strings. *Proc. VLDB Endow.*, 4(11):772–782, 2011.
- [40] Jian Gong, Reynold Cheng, and David W. Cheung. Efficient management of uncertainty in XML schema matching. *VLDB J.*, 21(3):385–409, 2012.
- [41] Sara Goodwin, John D. McPherson, and W. Richard McCombie. Coming of age: ten years of next-generation sequencing technologies. *Nature Reviews Genetics*, 17:333–351, 2016.
- [42] Szymon Grabowski and Marcin Raniszewski. Sampled suffix array with minimizers. *Softw. Pract. Exp.*, 47(11):1755–1771, 2017.
- [43] Manish Gupta and Michael Bendersky. Information retrieval with verbose queries. In Ricardo Baeza-Yates, Mounia Lalmas, Alistair Moffat, and Berthier A. Ribeiro-Neto, editors, *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval, Santiago, Chile, August 9-13, 2015*, pages 1121–1124. ACM, 2015.
- [44] Monika Rauch Henzinger. Finding near-duplicate web pages: a large-scale evaluation of algorithms. In Efthimis N. Efthimiadis, Susan T. Dumais, David Hawking, and Kalervo Järvelin, editors, *SIGIR 2006: Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, Seattle, Washington, USA, August 6-11, 2006*, pages 284–291. ACM, 2006.
- [45] Ming Hua, Jian Pei, and Xuemin Lin. Ranking queries on uncertain data. *VLDB J.*, 20(1):129–153, 2011.
- [46] Costas S. Iliopoulos, Christos Makris, Yannis Panagis, Katerina Perdikuri, Evangelos Theodoridis, and Athanasios K. Tsakalidis. The weighted suffix tree: An efficient data structure for handling molecular weighted sequences and its applications. *Fundam. Informaticae*, 71(2-3):259–277, 2006.
- [47] Chirag Jain, Arang Rhie, Nancy Hansen, Sergey Koren, and Adam M. Phillippy. Long-read mapping to repetitive reference sequences using winnowmap2. *Nat Methods*, 19:705–710, 2022.
- [48] Jeffrey Jestes, Feifei Li, Zhepeng Yan, and Ke Yi. Probabilistic string similarity joins. In Ahmed K. Elmagarmid and Divyakant Agrawal, editors, *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2010, Indianapolis, Indiana, USA, June 6-10, 2010*, pages 327–338. ACM, 2010.
- [49] Jiaojiao Jiang, Steve Versteeg, Jun Han, Md. Arafat Hossain, Jean-Guy Schneider, Christopher Leckie, and Zeinab Farahmandpour. P-gram: Positional n-gram for the clustering of machine-generated messages. *IEEE Access*, 7:88504–88516, 2019.
- [50] Dan Jurafsky and James H. Martin. *Speech and language processing: an introduction to natural language processing, computational linguistics, and speech recognition, 2nd Edition*. Prentice Hall series in artificial intelligence. Prentice Hall, Pearson Education International, 2009.
- [51] Bhargav Kanagal and Amol Deshpande. Indexing correlated probabilistic databases. In Ugur Çetintemel, Stanley B. Zdonik, Donald Kossmann, and Nesime Tatbul, editors, *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2009, Providence, Rhode Island, USA, June 29 - July 2, 2009*, pages 455–468. ACM, 2009.
- [52] Juha Kärrkäinen, Peter Sanders, and Stefan Burkhardt. Linear work suffix array construction. *J. ACM*, 53(6):918–936, 2006.
- [53] Richard M. Karp and Michael O. Rabin. Efficient randomized pattern-matching algorithms. *IBM J. Res. Dev.*, 31(2):249–260, 1987.
- [54] A.E. Kel, E. Gössling, I. Reuter, E. Cheremushkin, O.V. Kel-Margoulis, and E. Wingender. MATCHTM: a tool for searching transcription factor binding sites in DNA sequences. *Nucleic Acids Research*, 31(13):3576–3579, 07 2003.
- [55] Dominik Kempa and Tomasz Kociumaka. String synchronizing sets: sublinear-time BWT construction and optimal LCE data structure. In Moses Charikar and Edith Cohen, editors, *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, STOC 2019, Phoenix, AZ, USA, June 23-26, 2019*, pages 756–767. ACM, 2019.
- [56] Dominik Kempa and Tomasz Kociumaka. Breaking the $O(n)$ -barrier in the construction of compressed suffix arrays and suffix trees. In Nikhil Bansal and Viswanath Nagarajan, editors, *Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023*, pages 5122–5202. SIAM, 2023.
- [57] Tomasz Kociumaka, Solon P. Pissis, and Jakub Radoszewski. Pattern matching and consensus problems on weighted sequences and profiles. In Seok-Hee Hong, editor, *27th International Symposium on Algorithms and Computation, ISAAC 2016, December 12-14, 2016, Sydney, Australia*, volume 64 of *LIPIcs*, pages 46:1–46:12. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016.
- [58] Tomasz Kociumaka, Solon P. Pissis, and Jakub Radoszewski. Pattern matching and consensus problems on weighted sequences and profiles. *Theory Comput. Syst.*, 63(3):506–542, 2019.
- [59] Janne H. Korhonen, Petri Martinmäki, Cinzia Pizzi, Pasi Rastas, and Esko Ukkonen. MOODS: fast search for position weight matrix matches in DNA sequences. *Bioinform.*, 25(23):3181–3182, 2009.
- [60] Yuxuan Li, James Bailey, Lars Kulik, and Jian Pei. Efficient matching of substrings in uncertain sequences. In Mohammed Javeed Zaki, Zoran Obradovic, Pang-Ning Tan, Arindam Banerjee, Chandrika Kamath, and Srinivasan Parthasarathy, editors, *Proceedings of the 2014 SIAM International Conference on Data Mining, Philadelphia, Pennsylvania, USA, April 24-26, 2014*, pages 767–775. SIAM, 2014.
- [61] Glennis A. Logsdon, Mitchell R. Vollger, and Evan E. Eichler. Long-read human genome sequencing and its applications. *Nat. Rev. Genet.*, 21(10):597–614, 2020.
- [62] Grigoriou Loukides and Solon P. Pissis. Bidirectional string anchors: A new string sampling mechanism. In Petra Mutzel, Rasmus Pagh, and Grzegorz Herman, editors, *29th Annual European Symposium on Algorithms, ESA 2021, September 6-8, 2021, Lisbon, Portugal (Virtual Conference)*, volume 204 of *LIPIcs*, pages 64:1–64:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.
- [63] Veli Mäkinen and Gonzalo Navarro. Position-restricted substring searching. In José R. Correa, Alejandro Hevia, and Marcos A. Kiwi, editors, *LATIN 2006: Theoretical Informatics, 7th Latin American Symposium, Valdivia, Chile, March 20-24, 2006, Proceedings*, volume 3887 of *Lecture Notes in Computer Science*, pages 703–714. Springer, 2006.
- [64] Udi Manber and Eugene W. Myers. Suffix arrays: A new method for on-line string searches. *SIAM J. Comput.*, 22(5):935–948, 1993.
- [65] Olena Medelyan and Ian H. Witten. Thesaurus based automatic keyphrase indexing. In Gary Marchionini, Michael L. Nelson, and Catherine C. Marshall, editors, *ACM/IEEE Joint Conference on Digital Libraries, JCDL 2006, Chapel Hill, NC, USA, June 11-15, 2006, Proceedings*, pages 296–297. ACM, 2006.
- [66] Donald R. Morrison. PATRICIA - practical algorithm to retrieve information coded in alphanumeric. *J. ACM*, 15(4):514–534, 1968.
- [67] Ingo Müller, Cornelius Ratsch, and Franz Färber. Adaptive string dictionary compression in in-memory column-store database systems. In Sihem Amer-Yahia, Vassilis Christophides, Anastasios Kementsietsidis, Minos N. Garofalakis, Stratos Iliadis, and Vincent Leroy, editors, *Proceedings of the 17th International Conference on Extending Database Technology, EDBT 2014, Athens, Greece, March 24-28, 2014*, pages 283–294. OpenProceedings.org, 2014.
- [68] Claro Noda, Shashi Prabhu, Mário Alves, Thiemo Voigt, and Carlo Alberto Boano. CRAWDAD cister/rssi. <https://dx.doi.org/10.15783/C7WC75>, 2022.
- [69] Cinzia Pizzi, Pasi Rastas, and Esko Ukkonen. Finding significant matches of position weight matrices in linear time. *IEEE ACM Trans. Comput. Biol. Bioinform.*, 8(1):69–79, 2011.
- [70] Yinian Qi, Rohit Jain, Sarvjeet Singh, and Sunil Prabhakar. Threshold query optimization for uncertain data. In Ahmed K. Elmagarmid and Divyakant Agrawal, editors, *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2010, Indianapolis, Indiana, USA, June 6-10, 2010*, pages 315–326. ACM, 2010.
- [71] Niranjan Rai and Xiang Lian. Distributed probabilistic top-k dominating queries over uncertain databases. *Knowl. Inf. Syst.*, 65(11):4939–4965, 2023.
- [72] Michael Roberts, Wayne Hayes, Brian R. Hunt, Stephen M. Mount, and James A. Yorke. Reducing storage requirements for biological sequence comparison. *Bioinform.*, 20(18):3363–3369, 2004.
- [73] Patricia Rodriguez-Tomé, Peter Stoehr, Graham Cameron, and Tomas P. Flores. The european bioinformatics institute (EBI) databases. *Nucleic Acids Res.*, 24(1):6–12, 1996.
- [74] Saul Schleimer, Daniel Shawcross Wilkerson, and Alexander Aiken. Winnowing: Local algorithms for document fingerprinting. In Alon Y. Halevy, Zachary G. Ives, and AnHai Doan, editors, *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data, San Diego, California, USA, June 9-12, 2003*, pages 76–85. ACM, 2003.
- [75] Sarvjeet Singh, Chris Mayfield, Sunil Prabhakar, Rahul Shah, and Susanne E. Hambrusch. Indexing uncertain categorical data. In Rada Chirkova, Asuman Dogac, M. Tamer Özsu, and Timos K. Sellis, editors, *Proceedings of the 23rd International Conference on Data Engineering, ICDE 2007, The Marmara Hotel, Istanbul, Turkey, April 15-20, 2007*, pages 616–625. IEEE Computer Society, 2007.

- [76] Sarvjeet Singh, Chris Mayfield, Rahul Shah, Sunil Prabhakar, Susanne E. Hambrusch, Jennifer Neville, and Reynold Cheng. Database support for probabilistic attributes and tuples. In Gustavo Alonso, José A. Blakeley, and Arbee L. P. Chen, editors, *Proceedings of the 24th International Conference on Data Engineering, ICDE 2008, April 7-12, 2008, Cancún, Mexico*, pages 1053–1061. IEEE Computer Society, 2008.
- [77] Zitan Sun, Xin Huang, Jianliang Xu, and Francesco Bonchi. Efficient probabilistic truss indexing on uncertain graphs. In Jure Leskovec, Marko Grobelnik, Marc Najork, Jie Tang, and Leila Zia, editors, *WWW '21: The Web Conference 2021, Virtual Event / Ljubljana, Slovenia, April 19-23, 2021*, pages 354–366. ACM / IW3C2, 2021.
- [78] Yufei Tao, Reynold Cheng, Xiaokui Xiao, Wang Kay Ngai, Ben Kao, and Sunil Prabhakar. Indexing multi-dimensional uncertain data with arbitrary probability density functions. In Klemens Böhm, Christian S. Jensen, Laura M. Haas, Martin L. Kersten, Per-Åke Larson, and Beng Chin Ooi, editors, *Proceedings of the 31st International Conference on Very Large Data Bases, Trondheim, Norway, August 30 - September 2, 2005*, pages 922–933. ACM, 2005.
- [79] Yufei Tao, Xiaokui Xiao, and Reynold Cheng. Range search on multidimensional uncertain data. *ACM Trans. Database Syst.*, 32(3):15, 2007.
- [80] The Computational Pan-Genomics Consortium. Computational pan-genomics: status, promises and challenges. *Briefings in Bioinformatics*, 19(1):118–135, 2018.
- [81] Gerry Tonkin-Hill, Inigo Martincorena, Roberto Amato, Andrew RJ Lawson, Moritz Gerstung, Ian Johnston, David K Jackson, Naomi Park, Stefanie V Lensing, Michael A Quail, Sónia Gonçalves, Cristina Ariani, Michael Spencer Chapman, William L Hamilton, Luke W Meredith, Grant Hall, Aminu S Jahun, Yasmin Chaudhry, Myra Hosmillo, Malte L Pinckert, Iliana Georgana, Anna Yakovleva, Laura G Caller, Sarah L Caddy, Theresa Feltwell, Fahad A Khokhar, Charlotte J Houldcroft, Martin D Curran, Surendra Parmar, The COVID-19 Genomics UK (COG-UK) Consortium, Alex Alderton, Rachel Nelson, Ewan M Harrison, John Sillitoe, Stephen D Bentley, Jeffrey C Barrett, M Estee Torok, Ian G Goodfellow, Cordelia Langford, Dominic Kwiatkowski, and Wellcome Sanger Institute COVID-19 Surveillance Team. Patterns of within-host genetic diversity in SARS-CoV-2. *eLife*, 10:e66857, aug 2021.
- [82] Kazutoshi Umemoto, Ruihua Song, Jian-Yun Nie, Xing Xie, Katsumi Tanaka, and Yong Rui. Search by screenshots for universal article clipping in mobile apps. *ACM Trans. Inf. Syst.*, 35(4):34:1–34:29, 2017.
- [83] Angelos Vlavianos, Lap Kong Law, Ioannis Broustis, Srikanth V. Krishnamurthy, and Michalis Faloutsos. Assessing link quality in IEEE 802.11 wireless networks: Which is the right metric? In *Proceedings of the IEEE 19th International Symposium on Personal, Indoor and Mobile Radio Communications, PIMRC 2008, 15-18 September 2008, Cannes, French Riviera, France*, pages 1–6. IEEE, 2008.
- [84] Adrian Vogelsgesang, Michael Haubenschild, Jan Finis, Alfons Kemper, Viktor Leis, Tobias Mühlbauer, Thomas Neumann, and Manuel Then. Get real: How benchmarks fail to represent the real world. In Alexander Böhm and Tilmann Rabl, editors, *Proceedings of the 7th International Workshop on Testing Database Systems, DBTest@SIGMOD 2018, Houston, TX, USA, June 15, 2018*, pages 1:1–1:6. ACM, 2018.
- [85] Peter Weiner. Linear pattern matching algorithms. In *14th Annual Symposium on Switching and Automata Theory, Iowa City, Iowa, USA, October 15-17, 1973*, pages 1–11, 1973.
- [86] Aaron M. Wenger et al. Accurate circular consensus long-read sequencing improves variant detection and assembly of a human genome. *Nat. Biotechnol.*, 37:1155–1162, 2019.
- [87] Bohua Yang, Dong Wen, Lu Qin, Ying Zhang, Lijun Chang, and Rong-Hua Li. Index-based optimal algorithm for computing k-cores in large uncertain graphs. In *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8-11, 2019*, pages 64–75. IEEE, 2019.
- [88] Ke Yi, Feifei Li, George Kollios, and Divesh Srivastava. Efficient processing of top-k queries in uncertain databases with x-relations. *IEEE Trans. Knowl. Data Eng.*, 20(12):1669–1682, 2008.
- [89] Liming Zhan, Ying Zhang, Wenjie Zhang, and Xuemin Lin. Identifying top k dominating objects over uncertain data. In Sourav S. Bhowmick, Curtis E. Dyreson, Christian S. Jensen, Mong-Li Lee, Agus Muliantara, and Bernhard Thalheim, editors, *Database Systems for Advanced Applications - 19th International Conference, DASFAA 2014, Bali, Indonesia, April 21-24, 2014. Proceedings, Part I*, volume 8421 of *Lecture Notes in Computer Science*, pages 388–405. Springer, 2014.
- [90] Hongyu Zheng, Carl Kingsford, and Guillaume Marçais. Improved design and analysis of practical minimizers. *Bioinformatics*, 36(Supplement_1):i119–i127, 07 2020.